

CLEAN ARCHITECTURE

SOLID 원칙

3부: 설계 원칙

7장 ~ 11장

2026.04.10

“

*SOLID는 5개의 독립된 규칙이 아니다.
결국은 OCP를 위한 투쟁이다.*

— 7장~11장을 읽고 내린 결론

SRP: 누가 이 코드를 바꾸게 하는가?

OrderService.java

```
class OrderService {  
    int calculateOrderPrice(Order o) { ... }  
    int calculateSettlement(Order o) { ... }  
    Report generateReport(Order o) { ... }  
}
```

📌 프로덕트팀 → 쿠폰 적용 변경

📌 정산팀 → 수수료 기준 변경

📌 운영팀 → 리포트 필드 추가

정산팀: "수수료에서 배달비 빼주세요" → 개발자가 `getBaseAmount()` 수정

그런데 `calculateOrderPrice()`도 같은 함수를 쓰고 있었다

프로덕트팀: "저희 아무것도 안 바꿨는데 왜 금액이 달라졌죠?"

→ 액터별로 클래스를 분리하면 이 문제가 사라진다

OCP: 수정 없이 확장할 수 있는가?

수정이 필요한 설계

NotificationService.java

```
void notify(Order o, String type) {  
    if (type.equals("EMAIL")) {  
        // 이메일 로직  
    } else if (type.equals("SMS")) {  
        // SMS 로직  
    } else if (type.equals("KAKAO")) {  
        // 카카오톡 로직  
    }  
    // 슬랙 추가? → 여기를 수정  
}
```

확장만으로 해결되는 설계

NotificationService.java

```
interface Notifier {  
    void send(Order order);  
}  
  
class NotificationService {  
    private List<Notifier> notifiers;  
    void notify(Order o) {  
        for (Notifier n : notifiers)  
            n.send(o);  
    }  
}  
  
// 슬랙 추가? → 새 클래스만 추가
```

열려 있다 = 새 파일 추가 | 닫혀 있다 = 기존 파일 수정 불필요

LSP: 인터페이스를 믿을 수 있는가?

PaymentService.java

```
// NaverPayment은 금액을 "천원 단위"로 받는다
// → 호출하는 쪽에 if/else가 생긴다

if (gateway instanceof NaverPayment) {
    amount = amount / 1000; // LSP 위반!
}

gateway.pay(order, amount);
```

LSP 위반
구현체마다 행위 다름



if/else 등장
호출 측 분기 추가



OCP 위반
새 PG마다 기존 코드 수정



확장 구조 붕괴

구현체가 뭔지 **확인해야** 하는 순간 설계가 깨진 것이다

ISP: 불필요한 짐을 지고 있지 않은가?

안 쓰는 것에 의존하면, 그것이 변할 때 나도 영향받는다



클래스 수준

문제

Penguin이 fly()를 억지 구현

해결

역할별 인터페이스 분리



모듈 수준

문제

common-module 변경 시 전체 재빌드

해결

팀별 모듈 분리



시스템 수준

문제

안 쓰는 DB 드라이버의 보안 패치 피해

해결

인터페이스로 감싸거나 경량 대체

DIP: 추상에 의존하라

구체에 의존

```
OrderService.java

class OrderService {
    // 구체 클래스에 직접 의존
    private MySQLOrderRepository repo
        = new MySQLOrderRepository();
    private TossPaymentClient pay
        = new TossPaymentClient();
}
```

추상에 의존

```
OrderService.java

class OrderService {
    // 인터페이스에만 의존
    private final OrderRepository repo;
    private final PaymentGateway pay;
    // 구체는 외부에서 주입
    OrderService(OrderRepository r,
                  PaymentGateway p) {
        this.repo = r; this.pay = p;
    }
}
```

기준: "바뀌었을 때 감당할 수 없는가?" — 가능성이 아니라 리스크로 판단

결국은 OCP를 위한 투쟁

OCP — 수정 없이 확장

이것이 성립하려면

SRP

변경의 이유를
분리한다

LSP

대체 가능해야
확장이 작동

ISP

불필요한 의존을
차단한다

이 모든 것을 실현하는 메커니즘

DIP — 의존성 역전

추상에 의존하여 의존성 방향을 뒤집는다

같이 생각해볼 질문



추상화는 언제 해야 하는가?

"바뀔 가능성"으로 판단할 것인가?

"바뀌었을 때 감당할 수 없는가"로 판단할 것인가?

1%의 가능성에 대비하는 추상화는
99%의 시간 동안 불필요한 복잡성이다

같이 생각해볼 질문 2



SOLID 중 가장 핵심인 원칙은
무엇이라고 생각하시나요?

OCP — 나머지 4개는 이것을 위한 조건과 수단?

DIP — 이것 없이는 나머지가 무력화?

SRP — 이것이 먼저 지켜져야 나머지가 의미 있다?

정답은 없습니다. 여러분의 실무에서 가장 체감되는 원칙은?

SRP가 분리하고, LSP가 보장하고, ISP가 차단하고, DIP가 실현한다

다섯 원칙은 따로가 아니다.

모두 OCP를 위한 하나의 전략이다.

클린 아키텍처 3부: 7장 ~ 11장